

PAWS

Programmazione Mobile - Applicazione in Kotlin e Flutter



UNIVERSITÀ POLITECNICA DELLE MARCHE

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

RELATORI:

Emanuele Storti

TESINA DI:

Tarek Naja
Sara Vaccaro

A.A 2025/2026

Indice

INTRODUZIONE	1
Descrizione in linguaggio naturale.	1
GLOSSARIO DEI TERMINI	2
PROGETTAZIONE E SVILUPPO APPLICAZIONE	4
Analisi dei requisiti	4
Requisiti Funzionali	4
Requisiti Non Funzionali: Kotlin (App Nativa)	5
Requisiti Non Funzionali: Flutter (App Cross-Platform)	6
Descrizione dei Casi d'Uso	7
Area: Gestione Utente	7
Area: Gestione Cucciolo	14
Area: Gestione Interazioni	19
Area: Gestione Navigazione	24
Area: Gestione Notifiche	27
DATABASE E STORAGE	30
Database Remoto NoSQL (Cloud Firestore)	30
Cloud Object Storage (Supabase Storage)	30
Local Storage (SharedPreferences)	31
Sincronizzazione Offline e Persistenza Locale	31
Politiche di Sicurezza (Firestore Security Rules)	31
PROGETTAZIONE E SVILUPPO APPLICAZIONE IN KOTLIN	33
Architettura del sistema	33
Mappa globale dell'architettura	34
Architettura d'avvio e navigazione	34
Architettura gestione utente e autenticazione	36
Architettura gestione cuccioli e annunci	37
Architettura gestione notifiche e preferiti	38
Mockup	39
Verifica e Validazione del Software: Unit Tests	40
Test Manuali	40
Unit Test	40

Introduzione

Descrizione in linguaggio naturale

Il progetto consiste nello sviluppo di **Paws**, un'applicazione mobile pensata per amanti degli animali privati e rifugi. L'obiettivo dell'applicazione è offrire uno strumento per facilitare l'adozione e la condivisione di annunci relativi a cuccioli ed animali domestici, tenendo traccia delle schede di dettaglio dei singoli animali e consentendo il contatto diretto tra gli utenti.

Il sistema prevede due macro-tipologie di utenti:

- **Utenti privati:** individui che utilizzano l'applicazione per consultare gli annunci disponibili, salvare gli animali preferiti nella propria sezione dedicata, pubblicare nuovi annunci di adozione, filtrare la ricerca per razza, età, sesso e tipologia di utente, contattare direttamente i proprietari tramite chiamata telefonica, nonché cercare altri utenti e rifugi per animali (animal shelters) per visionarne i profili e i relativi post.
- **Rifugi e canili (Animal shelters):** strutture ed organizzazioni dedicate che dispongono di funzionalità per la pubblicazione e gestione di annunci di adozione su più ampia scala, promuovendo la visibilità degli animali in cerca di una casa e gestendo il proprio profilo informativo.

Sintesi dell'Approccio

Per la realizzazione del sistema, lo sviluppo dell'applicazione mobile è stato suddiviso in due percorsi complementari:

- **Applicazione Android nativa:** sviluppata in Kotlin sfruttando l'architettura nativa Android e la libreria di componenti UI, garantendo elevate prestazioni, gesture fluide (swipe-to-favorite) ed integrazione sia con i servizi di sistema (come l'avvio delle chiamate) sia con servizi di backend esterni. Nello specifico, sono stati utilizzati Firebase per l'autenticazione e la memorizzazione delle credenziali degli utenti e Supabase per il caricamento delle immagini e l'archiviazione dei dati completi relativi ai cuccioli.
- **Applicazione clone cross-platform:** realizzata in Flutter per consentire la portabilità del codice a partire da un'unica codebase, replicando fedelmente le funzionalità, l'identità visiva e l'interfaccia dell'applicazione nativa.

Questo approccio parallelo consente di analizzare e confrontare i vantaggi dello sviluppo nativo in termini di integrazione con il sistema operativo e risposta delle gesture rispetto ai benefici legati alla velocità di sviluppo e riusabilità offerti dalle moderne tecnologie cross-platform

Glossario dei Termini

Il seguente glossario definisce in modo univoco i termini di business e tecnologici utilizzati nell'ambito del progetto per evitare ambiguità durante le fasi di analisi, progettazione e sviluppo.

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Utente Privato	Persona fisica registrata che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Privato, User
Animal Shelter	Struttura d'accoglienza o associazione no-profit che utilizza l'app per cercare cuccioli, salvare preferiti e pubblicare annunci di adozione	BUSINESS	Rifugio, Ente di adozione
Cucciolo	Entità principale dell'applicazione che rappresenta un animale pubblicato con foto, nome, età, sesso, tipo e descrizione	BUSINESS	Animale, Post, Pet, Puppy
Scheda Dettaglio	Schermata informativa dedicata ad un singolo cucciolo contenente tutti i dettagli dell'animale e i contatti del proprietario	BUSINESS	Dettagli Cucciolo, Puppy Details
Preferiti	Sezione dell'applicazione in cui l'utente raccoglie e salva i cuccioli a cui è interessato	BUSINESS	Saved, Mi Piace, Favorites
Notifica	Messaggio automatico inviato al proprietario di un annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai preferiti	BUSINESS	Avviso, Notification
Filtro di Ricerca	Strumento per restringere l'elenco degli animali visualizzati in base a parametri quali età, sesso, tipo di animale e tipo di utente	BUSINESS	Criterio di Selezione, Search Filter

TERMINE	DESCRIZIONE	TIPO	SINONIMI
Account	Insieme delle credenziali di accesso (email e password) e dei dati personali che identificano un utente nel sistema	TECNICO	Profilo
Swipe to Favorite	Gesto di scorrimento orizzontale della scheda verso destra che consente di salvare un cucciolo nei preferiti	TECNICO	Gesture Swipe, Scorrimento Destra
Firebase	Servizio cloud utilizzato per la gestione della registrazione, dell'autenticazione sicura e della sessione degli utenti	TECNICO	Auth, Gestione Accessi
Cloud Firestore	Database NoSQL in tempo reale utilizzato per memorizzare i dati degli utenti, dei post, dei preferiti e delle notifiche	TECNICO	DB NoSQL, Firestore
Supabase	Servizio di Object Storage utilizzato per l'hosting e l'erogazione performante delle immagini del profilo e delle foto dei cuccioli	TECNICO	Storage Cloud

Progettazione e sviluppo applicazione

Nella presente sezione si analizzano in modo approfondito i requisiti del sistema, procedendo alla loro suddivisione secondo le seguenti categorie:

- **Requisiti funzionali**, che definiscono le specifiche funzionalità che il sistema è tenuto a fornire.
- **Requisiti non funzionali**, che individuano i vincoli e le qualità che il sistema deve soddisfare, quali ad esempio l'usabilità, la sicurezza e le prestazioni.

Analisi dei requisiti

Requisiti Funzionali

Area: Gestione Utente

- **RF1: Registrazione utente:** il sistema deve permettere la creazione di un account inserendo email, password, nome, cognome, città, provincia, CAP, numero di telefono, username e la selezione del tipo di account (Utente Privato o Animal Shelter).
- **RF2: Accesso utente:** il sistema deve consentire agli utenti registrati di accedere all'applicazione inserendo le proprie credenziali.
- **RF3: Modifica utente:** il sistema deve permettere all'utente di modificare i propri dati personali, aggiornare la foto profilo, richiedere il reset della password via email e modificare la tipologia di account.
- **RF4: Logout:** il sistema deve consentire all'utente di effettuare un logout sicuro dalla sessione attiva.
- **RF5: Elimina utente:** il sistema deve permettere all'utente di eliminare definitivamente il proprio account.
- **RF6: Visualizza utente:** il sistema deve permettere all'utente di visualizzare la pagina di profilo di un altro utente consultando le informazioni pubbliche e i post pubblicati.

Area: Gestione Cucciolo

- **RF7: Crea cucciolo:** il sistema deve permettere all'utente di caricare un nuovo annuncio specificando nome del cucciolo, tipo di animale (cane, gatto, uccello), sesso (maschio, femmina), età, didascalia descrittiva e foto dell'animale.
- **RF8: Modifica cucciolo:** il sistema deve permettere all'autore dell'annuncio di modificare le informazioni inserite.
- **RF9: Elimina cucciolo:** il sistema deve permettere all'autore dell'annuncio di eliminare definitivamente il post e la relativa immagine memorizzata nel cloud.
- **RF10: Visualizza post e scheda dettaglio:** il sistema deve mostrare i post dei cuccioli pubblicati e consentire la visualizzazione delle informazioni complete dell'animale e del proprietario.

Area: Gestione interazioni

- **RF11: Aggiungi ai preferiti:** il sistema deve permettere di aggiungere un cucciolo ai preferiti effettuando uno swipe verso destra dalla home o premendo sul cuore dal profilo di un Utente o Animal Shelter.
- **RF12: Rimuovi dai preferiti:** il sistema deve permettere di rimuovere un cucciolo dai preferiti premendo sul cuore nella lista dei preferiti.
- **RF13: Calcolo dei mi piace:** il sistema deve calcolare e mostrare in tempo reale il numero totale di salvataggi ricevuti per ciascun post.
- **RF14: Chiamata telefonica:** il sistema deve consentire di avviare una chiamata verso il numero di telefono del proprietario.

Area: Gestione Navigazione

- **RF15: Cerca:** il sistema deve permettere di cercare utenti o cuccioli inserendo il testo nella barra di ricerca.
- **RF16: Filtra:** il sistema deve consentire il filtraggio degli annunci in base a età, sesso, tipo di animale e tipo di utente.

Area: Gestione Notifiche

- **RF17: Genera notifica:** il sistema deve generare una notifica destinata al proprietario dell'annuncio quando un altro utente aggiunge/rimuove il suo cucciolo ai/dai preferiti.
- **RF18: Visualizza notifica:** il sistema deve mostrare all'utente un elenco delle notifiche ricevute.
- **RF19: Elimina notifica:** il sistema deve permettere all'utente l'eliminazione delle notifiche.

Requisiti Non Funzionali: Kotlin (App Nativa)

- **RNF1-K: Tecnologia di Sviluppo e SDK:** L'applicazione nativa è sviluppata in Kotlin con Android SDK, strutturata mediante Activity e Fragment per la navigazione tra le varie sezioni della schermata principale.
- **RNF2-K: Layout ed Interfaccia Grafica:** L'interfaccia utente è realizzata interamente mediante Layout XML, utilizzando i componenti Material Design e la libreria ViewBinding per il collegamento sicuro tra codice Kotlin e layout.
- **RNF3-K: Autenticazione e Database Cloud:** La gestione della sessione utente e la persistenza dei dati di dominio (Post, Profili Utente, Notifiche, Preferiti) sono gestite tramite Firebase Auth per l'autenticazione ed a Cloud Firestore per la lettura/scrittura in tempo reale dei dati.
- **RNF4-K: Cloud Storage per i Media:** L'upload e la gestione dello storage remoto delle immagini di profili e cuccioli sono realizzati tramite l'integrazione diretta con Supabase Storage.
- **RNF5-K: Rendering e Caching delle Immagini:** Il caricamento asincrono, il rendering ed il caching locale delle immagini da URL remoto nelle ImageView dell'interfaccia utente sono affidati alla libreria Glide.
- **RNF6-K: Visualizzazione Dati e RecyclerView:** L'applicazione organizza la presentazione delle liste e delle griglie di dati (post, cuccioli, notifiche, preferiti) mediante RecyclerView e Adapter dedicati (FeedAdapter, GridPostAdapter, FavoritesAdapter, NotificationAdapter, SearchAdapter).

Requisiti Non Funzionali: Flutter (App Cross-Platform)

- **RNF1-F: Tecnologia di Sviluppo Cross-Platform:** L'applicazione cross-platform è sviluppata in Dart con il framework Flutter (SDK ^3.12.2), garantendo la compatibilità sia per dispositivi Android sia per iOS con un'unica codebase.
- **RNF2-F: Servizi e Modelli Dati:** La gestione dei dati di dominio e della logica di backend è strutturata mediante Service dedicati (FirebaseService, SupabaseService) e Modelli di dati fortemente tipizzati (UserModel, PuppyPostModel, NotificationModel).
- **RNF3-F: Autenticazione e Cloud Firestore:** La gestione dell'autenticazione utente e le operazioni di lettura/scrittura sulle collezioni cloud (Cuccioli, Post, Preferiti, Notifiche, Utenti) sono affidate ai pacchetti ufficiali `firebase_auth` e `cloud_firestore`.
- **RNF4-F: Cloud Storage su Supabase:** Il caricamento e l'hosting remoto dei file multimediali (immagini dei cuccioli nel bucket `puppy-images` ed avatar utenti nel bucket `profile-images`) sono realizzati tramite il pacchetto `supabase_flutter`.
- **RNF5-F: Gestione dell'Interfaccia e dello Stato:** L'interfaccia utente e l'aggiornamento dello stato reattivo sono realizzati mediante i componenti nativi `StatefulWidget` e `StatelessWidget` di Flutter, con aggiornamento dinamico gestito tramite `setState()` e l'invocazione diretta dei Service.
- **RNF6-F: Design System e Caching Immagini:** L'applicazione impiega la classe centralizzata `AppTheme` (`lib/theme/app_theme.dart`) per la definizione di colori (`AppColors`), stili ed elementi grafici, e la libreria `cached_network_image` per la visualizzazione ottimizzata delle immagini di rete con placeholder di caricamento.

Descrizione dei Casi d'Uso

Di seguito sono descritti nel dettaglio i principali casi d'uso per l'applicazione.

Area: Gestione Utente

Questa area raggruppa tutti i casi d'uso relativi all'autenticazione e all'amministrazione del profilo personale. Definisce i processi tramite gli utenti possono registrarsi, accedere in modo sicuro tramite le proprie credenziali, aggiornare le informazioni anagrafiche e gestire l'intero ciclo di vita del proprio account, inclusa la disconnessione e l'eliminazione definitiva dei dati.

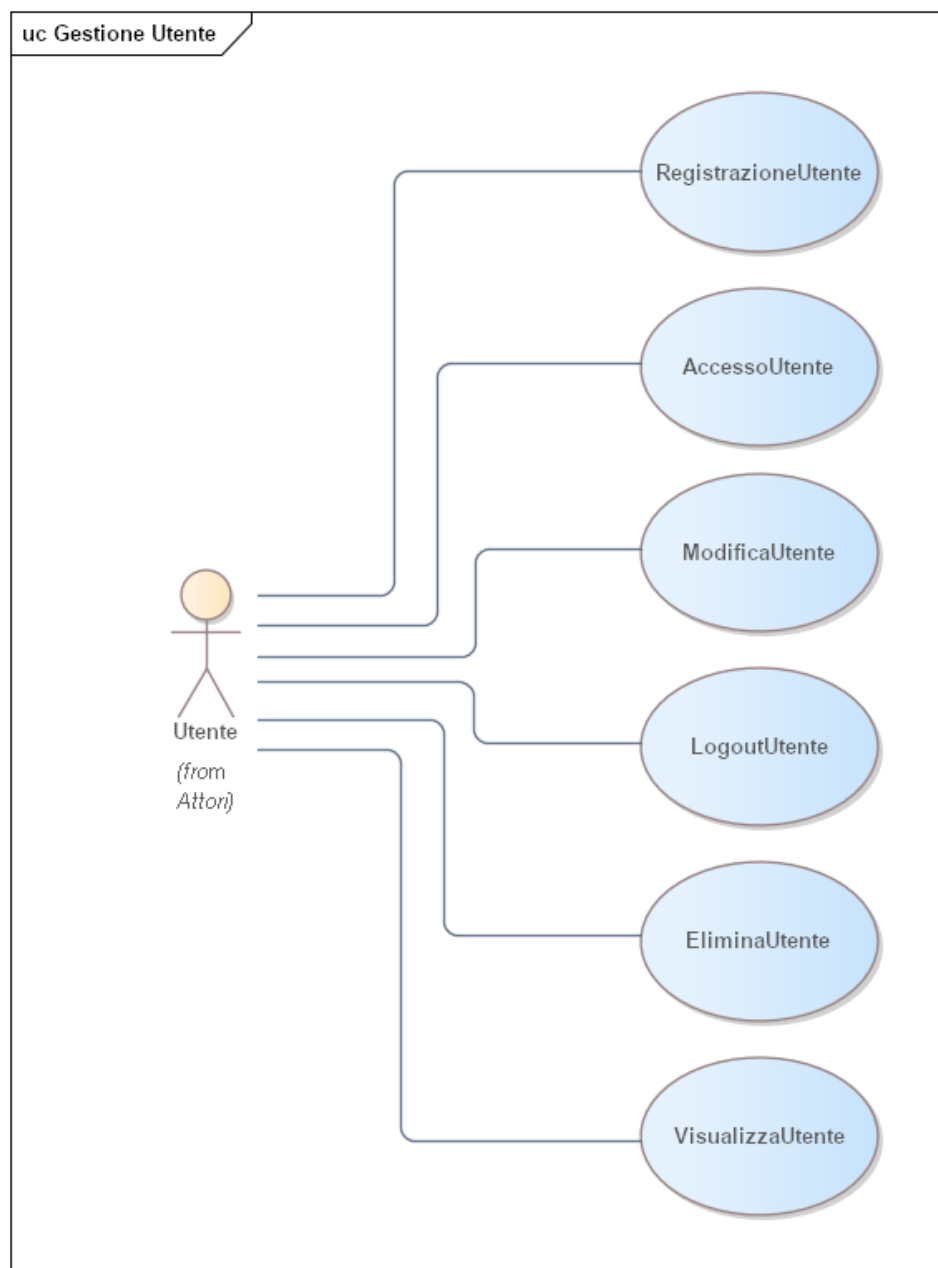


Figura 1: Gestione Utente

Caso d'uso: RegistrazioneUtente	
ID	UC-1
Breve descrizione	Un Utente crea un nuovo account per accedere all'applicazione.
Attori primari	Utente non autenticato
Attori secondari	Sistema di Autenticazione (Firebase Auth), Database (Cloud Firestore)
Precondizioni	1. L'applicazione è avviata e l'Utente si trova nella schermata di registrazione.
Sequenza degli eventi principale	1. L'Utente inserisce i propri dati: email, password, nome, cognome, città, provincia, CAP, numero di telefono e username. 2. L'Utente seleziona la tipologia di account (Utente Privato o Animal Shelter). 3. L'Utente convalida la registrazione premendo il pulsante dedicato. 4. Il sistema valida i dati inseriti. 5. Il sistema registra il nuovo account su Firebase Auth e ne salva le informazioni su Cloud Firestore. 6. L'Utente viene reindirizzato alla schermata principale e risulta autenticato.
Postcondizioni	1. L'account viene creato ed è abilitato all'uso, e l'Utente risulta autenticato nel sistema.
Sequenza degli eventi alternativa	1. If i dati non sono validi o l'email risulta già registrata, allora 1.1. Il sistema mostra un messaggio d'errore. 1.2. Il sistema invita l'Utente a correggere i campi errati o a inserire un indirizzo differente.

Caso d'uso: AccessoUtente	
ID	UC-2
Breve descrizione	Un Utente registrato effettua l'accesso all'applicazione inserendo le proprie credenziali.
Attori primari	Utente registrato
Attori secondari	Sistema di Autenticazione (Firebase Auth)
Precondizioni	1. L'Utente possiede un account valido e si trova nella schermata di login.
Sequenza degli eventi principale	1. L'Utente inserisce la propria email e password. 2. L'Utente preme il pulsante di accesso. 3. Il sistema verifica la correttezza delle credenziali. 4. Il sistema autentica l'Utente e lo reindirizza alla schermata principale.
Postcondizioni	1. L'Utente è autenticato con successo e può accedere alle funzionalità riservate del sistema.
Sequenza degli eventi alternativa	1. If le credenziali inserite risultano errate o inesistenti, allora 1.1. Il sistema mostra un messaggio d'errore per credenziali non valide. 1.2. Il modulo viene pulito o l'Utente è invitato a riprovare l'inserimento.

Caso d'uso: ModificaUtente	
ID	UC-3
Breve descrizione	L'Utente visualizza e aggiorna i propri dati personali o la foto profilo.
Attori primari	Utente autenticato
Attori secondari	Database (Cloud Firestore), Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato e si trova nella sezione di gestione del proprio profilo.
Sequenza degli eventi principale	1. Il sistema mostra i dati attuali precompilati nei rispettivi campi. 2. L'Utente modifica le informazioni desiderate. 3. Il sistema valida i nuovi dati. 4. Il sistema aggiorna le informazioni nel database.
Postcondizioni	1. I dati dell'account Utente sono aggiornati correttamente nel sistema.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: Logout	
ID	UC-4
Breve descrizione	L'Utente termina la propria sessione attiva in modo sicuro.
Attori primari	Utente autenticato
Attori secondari	Sistema di Autenticazione (Firebase Auth)
Precondizioni	1. L'Utente è attualmente autenticato nell'applicazione.
Sequenza degli eventi principale	1. L'Utente naviga nelle impostazioni del profilo e seleziona l'opzione di logout. 2. Il sistema revoca i token di accesso e distrugge la sessione attiva. 3. L'Utente viene reindirizzato alla schermata iniziale.
Postcondizioni	1. La sessione è terminata e non è più possibile accedere ai dati protetti senza effettuare un nuovo accesso.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: EliminaUtente	
ID	UC-5
Breve descrizione	L'Utente richiede l'eliminazione definitiva del proprio account e di tutti i dati associati.
Attori primari	Utente autenticato
Attori secondari	Firebase Auth, Database (Cloud Firestore), Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato e si trova nella sezione di gestione account.
Sequenza degli eventi principale	1. L'Utente seleziona l'opzione per eliminare l'account. 2. Il sistema mostra un avviso indicando che l'operazione è irreversibile e richiede un'ulteriore conferma di sicurezza. 3. L'Utente fornisce la conferma richiesta. 4. Il sistema procede all'eliminazione dei dati, cancellando le immagini collegate su Supabase Storage, rimuovendo i post, i preferiti e il profilo da Cloud Firestore, ed eliminando l'account da Firebase Auth. 5. L'Utente viene disconnesso e reindirizzato alla schermata iniziale mostrando un messaggio di conferma.
Postcondizioni	1. L'account e tutti i dati ad esso correlati sono rimossi dal sistema.
Sequenza degli eventi alternativa	1. If l'Utente annulla l'operazione nella finestra di conferma, allora 1.1. Il sistema interrompe la procedura di eliminazione. 1.2. L'Utente viene riportato alle impostazioni del profilo senza modifiche ai dati.

Caso d'uso: VisualizzaUtente	
ID	UC-6
Breve descrizione	L'Utente consulta la pagina di profilo di un altro utente visualizzarne le informazioni pubbliche e i post pubblicati.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore, Supabase
Precondizioni	1. L'Utente si trova nella scheda di dettaglio di un cucciolo.
Sequenza degli eventi principale	1. L'Utente seleziona il profilo del proprietario presente nella scheda del cucciolo. 2. Il sistema richiede al database i dati del profilo e l'elenco dei cuccioli pubblicati da quell'utente. 3. Il sistema recupera con successo le informazioni. 4. Il sistema mostra a schermo la pagina del profilo con i relativi dettagli e post.
Postcondizioni	1. L'Utente visualizza a schermo le informazioni e i post dell'Utente selezionato.
Sequenza degli eventi alternativa	1. If l'Utente cerca il profilo nella barra apposita della schermata principale, allora 1.1. L'Utente seleziona il profilo desiderato dall'elenco dei risultati. 1.2. Il sistema richiede i dati al database e mostra a schermo la pagina del profilo.

Area: Gestione Cucciolo

Quest'area raggruppa tutti i casi d'uso relativi alla gestione dei post di adozione. Definisce i processi tramite i quali gli utenti possono pubblicare nuovi post relativi ai cuccioli caricandone foto e dettagli, modificare o rimuovere le proprie inserzioni e, infine, di consultare le schede informative degli animali disponibili.

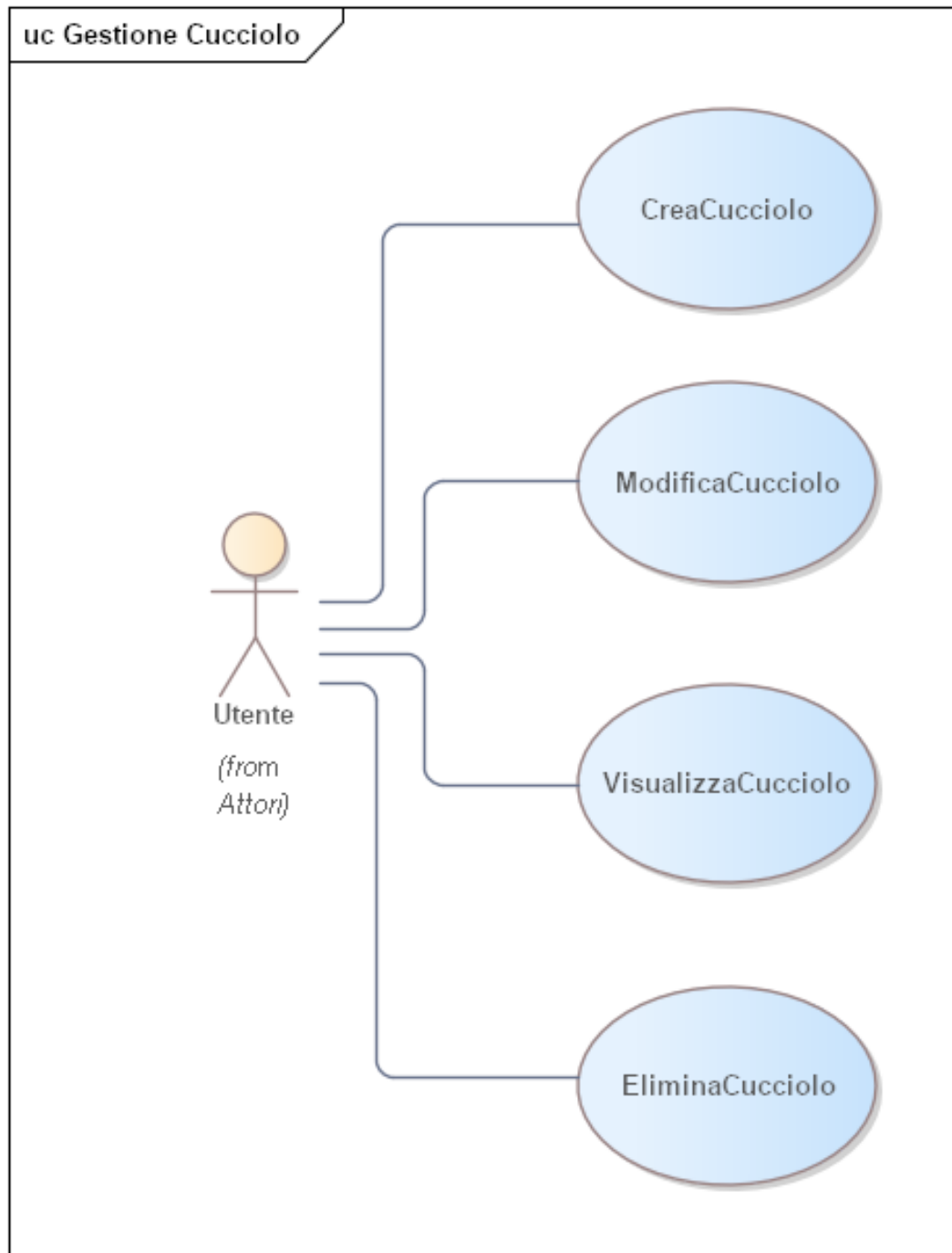


Figura 2: Gestione Cucciolo

Caso d'uso: CreaCucciolo	
ID	UC-7
Breve descrizione	Un Utente registrato crea e pubblica un nuovo post relativo a un cucciolo fornendo le informazioni richieste e caricando l'immagine.
Attori primari	Utente registrato
Attori secondari	Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato e si trova nella schermata di creazione di un nuovo post.
Sequenza degli eventi principale	1. L'Utente inserisce le informazioni del cucciolo e seleziona l'immagine. 2. L'Utente preme il pulsante di pubblicazione. 3. Il sistema salva il post e i file multimediali nel database. 4. Il sistema mostra un messaggio di conferma dell'avvenuta creazione e reindirizza l'Utente alla lista cuccioli.
Postcondizioni	1. Il nuovo post del cucciolo è salvato nel sistema ed è visibile all'interno dell'applicazione.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: ModificaCucciolo	
ID	UC-8
Breve descrizione	Un Utente registrato aggiorna le informazioni o l'immagine di un post precedentemente inserito.
Attori primari	Utente registrato (proprietario del post)
Attori secondari	Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato, è il proprietario del post e si trova nella schermata di modifica del cucciolo selezionato.
Sequenza degli eventi principale	1. Il sistema precompila il modulo con i dati e i file attualmente presenti. 2. L'Utente modifica uno o più campi. 3. L'Utente preme il pulsante di salvataggio delle modifiche. 4. Il sistema verifica la correttezza dei dati aggiornati. 5. Il sistema aggiorna le informazioni e le immagini sul database. 6. Il sistema mostra un messaggio di successo e reindirizza l'Utente alla lista cuccioli.
Postcondizioni	1. Le informazioni del post del cucciolo risultano aggiornate con successo nel database.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: EliminaCucciolo	
ID	UC-9
Breve descrizione	Un Utente registrato rimuove in modo permanente un post di un cucciolo di sua proprietà dal sistema.
Attori primari	Utente registrato (proprietario del post)
Attori secondari	Cloud Storage (Supabase Storage)
Precondizioni	1. L'Utente è autenticato, è il proprietario del post, e si trova nella lista dei propri post.
Sequenza degli eventi principale	1. L'Utente seleziona l'opzione per eliminare il post del cucciolo. 2. Il sistema richiede una conferma all'Utente prima di procedere. 3. L'Utente conferma l'eliminazione. 4. Il sistema rimuove definitivamente il post e le relative immagini dal database. 5. Il sistema mostra un messaggio di conferma dell'eliminazione.
Postcondizioni	1. Il post del cucciolo è rimosso dal database e non è più visibile o ricercabile nell'applicazione.
Sequenza degli eventi alternativa	1. If l'Utente annulla la richiesta nella finestra di conferma, allora 1.1. Il sistema chiude il dialogo di conferma senza cancellare i dati. 1.2. Il post del cucciolo rimane inalterato nel sistema.

Caso d'uso: VisualizzaCucciolo	
ID	UC-10
Breve descrizione	Un Utente consulta la scheda di dettaglio di un cucciolo per visualizzarne le informazioni e le foto.
Attori primari	Utente registrato
Attori secondari	Supabase Storage
Precondizioni	1. L'Utente si trova nella schermata principale o nel profilo di un utente.
Sequenza degli eventi principale	1. L'Utente preme sulle informazioni del cucciolo. 2. Il sistema richiede al database i dati dettagliati e i contenuti multimediali del cucciolo. 3. Il sistema recupera con successo le informazioni. 4. Il sistema mostra la schermata con i dettagli del cucciolo.
Postcondizioni	1. L'Utente visualizza a schermo le informazioni dettagliate del cucciolo richiesto.
Sequenza degli eventi alternativa	1. If l'Utente si trova nella lista dei preferiti, allora 1.1. L'Utente preme sull'anteprima del cucciolo per accedere ai dettagli. 1.2. Il sistema richiede al database i dati 1.3. Il sistema mostra la schermata con i dettagli del cucciolo.

Area: Gestione Interazioni

Quest'area raggruppa i casi d'uso relativi all'interazione con i post e con i proprietari dei cuccioli. Definisce i processi tramite i quali gli utenti possono salvare gli animali nella propria lista dei preferiti o avviare rapidamente un chiamata con altri utenti o rifugi, e infine descrive il meccanismo automatico con cui il sistema calcola e aggiorna il numero totale di "Mi piace" ricevuti da ciascun annuncio.

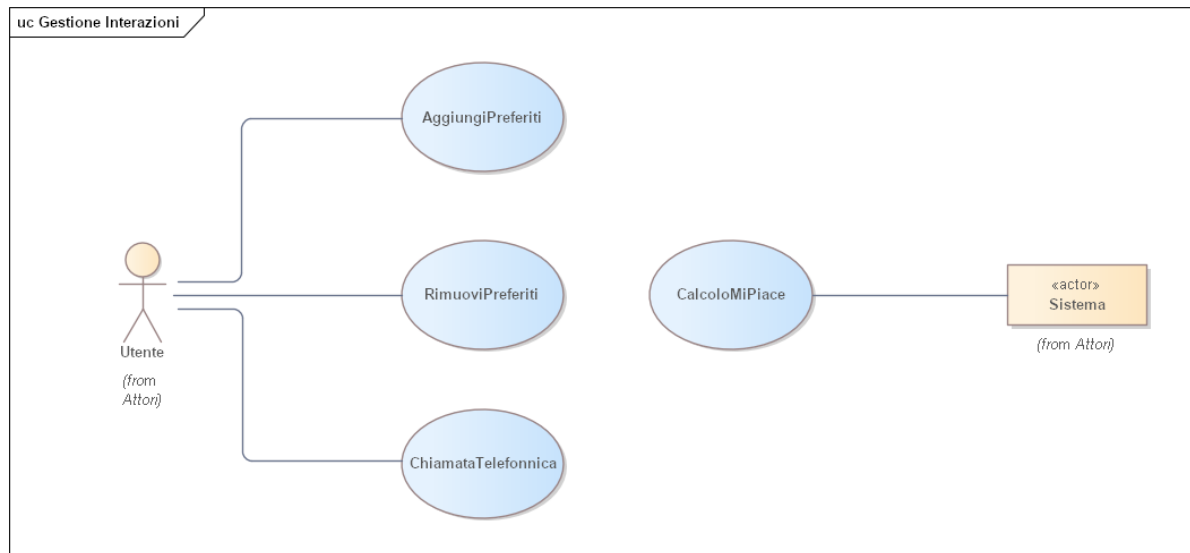


Figura 3: Gestione Interazioni

Caso d'uso: AggiungiPreferiti	
ID	UC-11
Breve descrizione	Un Utente registrato aggiunge un cucciolo alla propria lista dei preferiti.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente è autenticato e si trova nella schermata principale.
Sequenza degli eventi principale	1. L'Utente aggiunge un cucciolo ai preferiti effettuando un gesto di scorrimento verso destra (Swipe to Favorite) sulla scheda del cucciolo. 2. Il sistema memorizza l'associazione tra l'Utente e il cucciolo nel database. 3. Il sistema aggiorna l'interfaccia visiva per confermare l'avvenuto salvataggio.
Postcondizioni	1. Il cucciolo risulta salvato nella lista dei preferiti dell'Utente.
Sequenza degli eventi alternativa	1. If l'operazione viene eseguita dalla pagina di profilo di un altro Utente, allora 1.1. L'Utente seleziona l'icona dei preferiti (cuore) posta sull'anteprima del cucciolo. 1.2. Il sistema registra l'aggiunta ai preferiti nel database. 1.3. Il sistema aggiorna l'interfaccia attivando visivamente l'icona.

Caso d'uso: RimuoviPreferiti	
ID	UC-12
Breve descrizione	Un Utente registrato rimuove un cucciolo precedentemente salvato dalla propria lista dei preferiti.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente è autenticato e si trova nella sezione dei preferiti.
Sequenza degli eventi principale	1. L'Utente rimuove un cucciolo dai preferiti selezionando l'icona del cuore sull'anteprima del cucciolo. 2. Il sistema rimuove l'associazione tra l'Utente e il cucciolo nel database. 3. Il sistema aggiorna l'interfaccia visiva per confermare l'avvenuta rimozione.
Postcondizioni	1. Il cucciolo è rimosso definitivamente dalla lista dei preferiti dell'Utente.
Sequenza degli eventi alternativa	1. If l'operazione viene eseguita dalla pagina di profilo di un altro Utente, allora 1.1. L'Utente seleziona l'icona dei preferiti posta sull'anteprima del cucciolo. 1.2. Il sistema registra la rimozione dai preferiti nel database. 1.3. Il sistema aggiorna l'interfaccia disattivando visivamente l'icona.

Caso d'uso: CalcoloMiPiace	
ID	UC-13
Breve descrizione	Il sistema calcola e aggiorna il numero totale di "Mi piace" ricevuti dal post di un cucciolo.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente sta visualizzando il post di un cucciolo nella schermata principale.
Sequenza degli eventi principale	1. Il sistema richiede al database il totale delle interazioni registrate per il post. 2. Il database restituisce il conteggio aggiornato. 3. Il sistema calcola la somma totale e la formatta per la visualizzazione. 4. Il sistema mostra a schermo il numero totale di "Mi piace" accanto all'apposita icona.
Postcondizioni	1. Il totale dei "Mi piace" viene correttamente calcolato e mostrato nell'interfaccia del post.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: Chiamata Telefonica	
ID	UC-14
Breve descrizione	L'Utente avvia direttamente una chiamata telefonica verso il recapito del proprietario del cucciolo.
Attori primari	Utente
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente visualizza la scheda di dettaglio di un cucciolo in cui è presente il recapito di contatto.
Sequenza degli eventi principale	1. L'Utente preme il numero di telefono nella scheda del cucciolo. 2. Il sistema recupera il numero di telefono del proprietario associato al post. 3. Il sistema richiama le funzionalità del sistema operativo dello smartphone 4. Il sistema chiede conferma per l'avvio della chiamata
Postcondizioni	1. L'Utente avvia la chiamata verso il proprietario.
Sequenza degli eventi alternativa	Nessuna

Area: Gestione Navigazione

Quest'area raggruppa le funzionalità tutti i casi d'uso relativi all'esplorazione del database dei post. Definisce i processi tramite i quali gli utenti possono effettuare ricerche testuali mirate tramite parole chiave e di applicare filtri avanzati per restringere i risultati mostrati a schermo, facilitando così l'individuazione del cucciolo più in linea con le proprie preferenze.

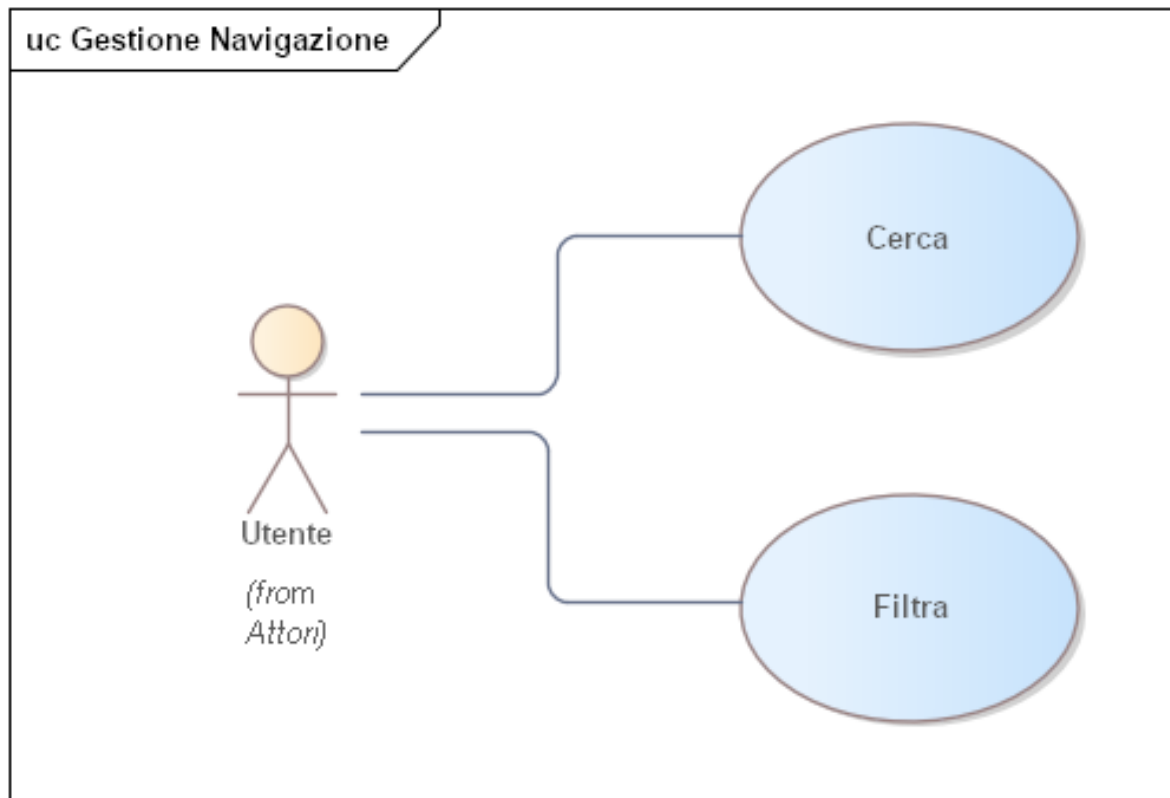


Figura 4: Gestione Navigazione

Caso d'uso: Cerca	
ID	UC-15
Breve descrizione	L'Utente inserisce una o più parole chiave per cercare utenti o cuccioli nell'applicazione.
Attori primari	Utente (registrato o non registrato)
Attori secondari	Database Remoto (es. Firebase Firestore / Supabase)
Precondizioni	1. L'Utente si trova nella schermata principale.
Sequenza degli eventi principale	1. L'Utente digita la parola chiave nella barra di ricerca. 2. L'Utente avvia la ricerca. 3. Il sistema richiede al database i dati che corrispondono al testo inserito. 4. Il sistema recupera i dati e mostra a schermo l'elenco dei risultati trovati.
Postcondizioni	1. L'applicazione mostra la lista degli utenti o dei cuccioli corrispondenti al testo cercato.
Sequenza degli eventi alternativa	1. If l'Utente si trova nella schermata dei propri post o nella lista dei preferiti, allora 1.1. L'Utente digita la parola chiave nell'apposita barra di ricerca locale. 1.2. Il sistema filtra l'elenco specifico richiedendo al database i risultati pertinenti. 1.3. Il sistema mostra a schermo l'elenco dei cuccioli filtrati.

Caso d'uso: Filtra	
ID	UC-16
Breve descrizione	L'Utente seleziona determinati parametri per restringere la lista degli utenti o cuccioli da visualizzare.
Attori primari	Utente (registrato o non registrato)
Attori secondari	Firebase Firestore, Supabase
Precondizioni	1. L'Utente è nella schermata principale ed apre il menu dei filtri.
Sequenza degli eventi principale	1. L'Utente seleziona uno o più criteri di filtraggio. 2. L'Utente preme il pulsante per applicare i filtri. 3. Il sistema richiede al database unicamente i post che soddisfano tutti i parametri selezionati. 4. Il sistema aggiorna l'interfaccia mostrata all'Utente con la nuova lista filtrata.
Postcondizioni	1. L'elenco visualizzato contiene esclusivamente gli utenti o i cuccioli che rispettano i filtri applicati.
Sequenza degli eventi alternativa	1. If l'Utente si trova nella sezione dei propri post o nella lista dei preferiti, allora 1.1. L'Utente apre il menu dei filtri e seleziona i criteri desiderati. 1.2. Il sistema filtra l'elenco specifico (post personali o preferiti) richiedendo al database solo i risultati pertinenti. 1.3. Il sistema mostra a schermo l'elenco filtrato.

Area: Gestione Notifiche

Quest'area raggruppa i casi d'uso dedicati al sistema di comunicazione e di avvisi in tempo reale. Gestisce la generazione e l'invio automatico delle notifiche push a fronte di eventi specifici di interesse per l'utente, consentendo inoltre la consultazione della cronologia degli avvisi e l'eliminazione di quest'ultimi per mantenere pulito il proprio centro notifiche.

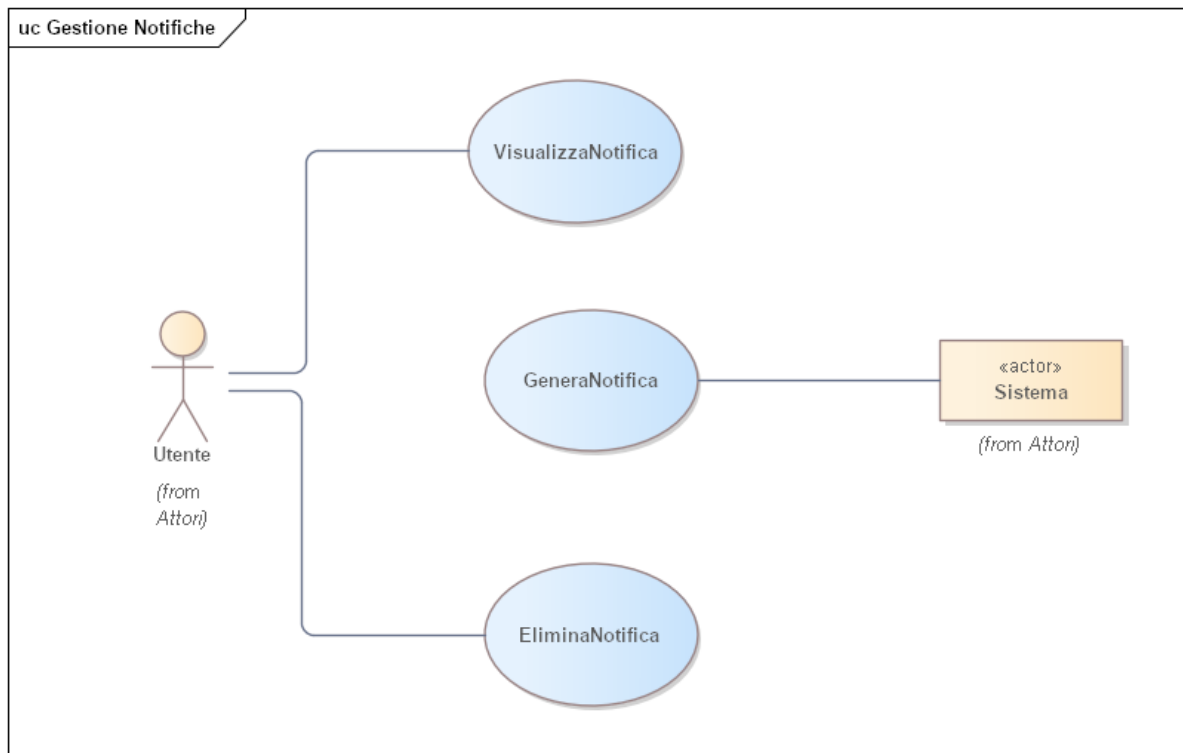


Figura 5: Gestione Notifiche

Caso d'uso: GeneraNotifica	
ID	UC-17
Breve descrizione	Il sistema genera e invia automaticamente una notifica all'Utente al verificarsi di un evento di suo interesse.
Attori primari	Sistema
Attori secondari	Firebase Cloud Messaging
Precondizioni	1. Si verifica un evento nel sistema che richiede l'invio di una notifica a un Utente destinatario.
Sequenza degli eventi principale	1. Il sistema rileva l'evento scatenante. 2. Il sistema compone il titolo, il testo e i dati della notifica. 3. Il sistema salva la notifica nel database dell'Utente destinatario. 4. Il sistema invia la notifica push al dispositivo dell'Utente tramite il servizio dedicato.
Postcondizioni	1. La notifica viene registrata nella sezione Notifiche e recapitata al suo dispositivo.
Sequenza degli eventi alternativa	Nessuna

Caso d'uso: VisualizzaNotifica	
ID	UC-18
Breve descrizione	L'Utente registrato accede al centro notifiche, consulta l'elenco delle notifiche ricevute.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente è autenticato e seleziona la sezione "Notifiche" dell'applicazione.
Sequenza degli eventi principale	1. L'Utente entra nella schermata delle notifiche. 2. Il sistema richiede al database la lista delle notifiche dell'Utente. 3. Il sistema mostra le notifiche ordinate dalla più recente.
Postcondizioni	1. L'Utente prende visualizza la notifica.
Sequenza degli eventi alternativa	1. If non sono presenti notifiche nell'account dell'Utente, allora 1.1. Il sistema mostra una schermata vuota informativa.

Caso d'uso: EliminaNotifica	
ID	UC-19
Breve descrizione	L'Utente registrato le notifiche dal proprio elenco.
Attori primari	Utente registrato
Attori secondari	Firebase Firestore
Precondizioni	1. L'Utente è autenticato e si trova all'interno della schermata "Notifiche".
Sequenza degli eventi principale	1. L'Utente seleziona l'opzione di eliminazione. 2. Il sistema rimuove la notifica dal database. 3. Il sistema aggiorna l'interfaccia rimuovendo la notifica dall'elenco.
Postcondizioni	1. La notifica eliminata viene cancellata permanentemente dal database dell'Utente.
Sequenza degli eventi alternativa	Nessuna

Database e Storage

La gestione dei dati nell'applicazione **Paws** adotta un'architettura distribuita e multi-database, progettata per garantire elevata reattività, sicurezza e scalabilità. A differenza di un approccio monolitico basato su un unico motore relazionale locale o cloud, Paws integra in modo sinergico diverse tecnologie specializzate di database e storage, sia per l'ambiente nativo Android in Kotlin che per la versione cross-platform in Flutter:

- **Database Cloud NoSQL (Firebase Cloud Firestore):** gestisce la persistenza dei dati di dominio in tempo reale (utenti, annunci di cuccioli, preferiti e notifiche).
- **Cloud Object Storage (Supabase Storage):** gestisce l'archiviazione efficiente dei file multimediali ad alta risoluzione (immagini del profilo e foto degli annunci).
- **Database di Autenticazione (Firebase Auth):** memorizza le credenziali e gestisce l'identità degli utenti generando gli identificatori univoci (UID).
- **Local Storage (SharedPreferences):** memorizza in locale piccoli quantitativi di dati a chiave-valore per lo stato della sessione e le preferenze grafiche.

Database Remoto NoSQL (Cloud Firestore)

Il database principale dell'applicazione è **Firebase Cloud Firestore**, un database documentale NoSQL basato su collezioni e documenti JSON-like. Firestore è stato scelto per la sua capacità di offrire la sincronizzazione in tempo reale tramite *WebSocket* e *Real-Time Listeners*, consentendo all'interfaccia utente di aggiornarsi istantaneamente al variare dei dati sul cloud.

Il database è organizzato nelle seguenti collezioni principali:

- **users:** contiene i profili informativi degli utenti e delle strutture di adozione (rifugi/canili). Ogni documento ha come ID il campo `uid` derivato da Firebase Auth e comprende: `name`, `surname`, `email`, `phone`, `role` (privato o rifugio), `address`, `description` e `avatarUrl`.
- **posts:** raccoglie gli annunci dei cuccioli in cerca di casa. Ciascun documento memorizza: `postId`, `uid` (proprietario dell'annuncio), `name` del cucciolo, `type` (specie animale), `breed` (razza), `age`, `sex`, `caption` (descrizione), `imageUrl` (link Supabase) e `timestamp` di creazione.
- **favorites:** rappresenta la relazione molti-a-molti tra gli utenti e i post salvati nei preferiti. Mantiene l'associazione `userId` – `postId`, consentendo il recupero rapido nella schermata dedicata e supportando le gesture native di *Swipe-to-Favorite*.
- **notifications:** traccia le notifiche di sistema ed interazione inviate agli utenti. Registra l'ID del destinatario (`recipientId`), l'ID del mittente (`senderId`), il post di riferimento, il tipo di notifica, lo stato di lettura (`isRead`) e il marcatore temporale.

Cloud Object Storage (Supabase Storage)

Per evitare l'appesantimento del database NoSQL con stringhe Base64 o oggetti binari di grandi dimensioni (BLOB), l'architettura di Paws disaccoppia la gestione dei dati testuali da quella dei contenuti multimediali affidandosi a **Supabase Storage**.

Supabase mette a disposizione bucket di storage dedicati:

- **Bucket profiles:** dedicato alle immagini di profilo degli utenti e dei canili.
- **Bucket puppies:** dedicato al caricamento delle foto dei cuccioli associate agli annunci.

Quando un utente carica un'immagine (tramite la `CreatePostFragment` in Kotlin o l'equivalente in Flutter), il file binario viene inviato tramite le API REST/SDK di Supabase Storage. Supabase restituisce un URL pubblico sicuro, il quale viene poi salvato nel relativo documento Firestore (`imageUrl` o `avatarUrl`). Lato client, le librerie di visualizzazione quali **Glide** (in Android Kotlin) e `cached_network_image` (in Flutter) caricano le immagini in modo asincrono sfruttando tecniche di caching in memoria e su disco.

Local Storage (SharedPreferences)

Accanto ai database di rete cloud, l'applicazione sfrutta il meccanismo di memorizzazione locale a chiave-valore offerto da **SharedPreferences** (in Android Native) e dal pacchetto `shared_preferences` (in Flutter).

Questo storage locale viene impiegato principalmente per:

- Salvare lo stato della sessione e l'ID utente corrente (UID) dopo il primo accesso.
- Memorizzare le preferenze grafiche dell'interfaccia utente (come la scelta del tema chiaro o scuro).
- Salvare temporaneamente il flag di completamento dell'onboarding.

In questo modo, all'avvio dell'applicazione è possibile determinare istantaneamente se reindirizzare l'utente alla schermata di Login o caricare la Home, evitando schermate di caricamento vuote e senza attendere l'inizializzazione dei database cloud o delle chiamate di rete.

Sincronizzazione Offline e Persistenza Locale

Una delle caratteristiche fondamentali dell'architettura multi-database di Paws è la capacità di operare anche in assenza temporanea di connettività internet. A differenza delle architetture tradizionali basate su SQL che richiedono la scrittura manuale di tabelle di sync ed attributi come `isSynced`, **Firestore** integra nativamente un motore di persistenza locale basato su cache interna.

Il funzionamento del meccanismo di sincronizzazione offline segue questo comportamento:

- **Scrittura offline:** quando l'utente crea un annuncio o aggiunge un cucciolo ai preferiti senza rete internet, lo SDK di Firestore memorizza l'operazione nel database SQLite/LevelDB locale del dispositivo e aggiorna immediatamente la UI (*Optimistic UI Updates*).
- **Ripristino connettività:** al rilevamento del segnale di rete, lo SDK invia in automatico le modifiche in coda ai server cloud Firebase, garantendo l'allineamento definitivo dei dati in background.

Politiche di Sicurezza (Firestore Security Rules)

Per proteggere l'integrità dei dati salvati nei database cloud e prevenire accessi o modifiche non autorizzate, sono state configurate apposite regole di sicurezza dichiarative su Cloud Firestore (*Security Rules*):

```

1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4
5     // Regole per la collezione utenti
6     match /users/{userId} {
7       allow read: if request.auth != null;
8       allow create, update: if request.auth != null && request.auth.uid ==
          userId;
9       allow delete: if request.auth != null && request.auth.uid == userId;
10    }
11
12    // Regole per la collezione post di adozione cuccioli
13    match /posts/{postId} {
14      allow read: if request.auth != null;
15      allow create: if request.auth != null && request.resource.data.uid ==
          request.auth.uid;

```

```
16     allow update, delete: if request.auth != null && resource.data.uid ==
17         request.auth.uid;
18 }
19 // Regole per la collezione dei preferiti
20 match /favorites/{favoriteId} {
21     allow read, write: if request.auth != null && resource.data.userId ==
22         request.auth.uid;
23 }
24 // Regole per la collezione notifiche
25 match /notifications/{notificationId} {
26     allow read, update, delete: if request.auth != null && resource.data.
27         recipientId == request.auth.uid;
28     allow create: if request.auth != null;
29 }
30 }
```

Unit Test 1: Regole di Sicurezza Cloud Firestore

Progettazione e sviluppo applicazione in Kotlin

Architettura del sistema

L'applicazione Android segue le linee guida di Google per l'architettura delle applicazioni moderne (Modern Android Architecture) basandosi sui principi della **Clean Architecture** e sul concetto fondamentale di "**Separation of Concerns**" (separazione delle responsabilità). Ciò consente di garantire un'elevata, manutenibilità e facilità di estensione delle componenti:

- **Data Layer:** gestisce l'accesso, la persistenza e la sincronizzazione remota dei dati. Interagisce con i servizi cloud attraverso l'integrazione di **Firebase Cloud Firestore** utilizzato per la sincronizzazione asincrona di annunci, notifiche e preferiti, **Firebase Authentication** per la gestione sicura delle sessioni utente, e **Supabase Client** per il caricamento efficiente delle immagini. Include inoltre la libreria **Glide** per il caching e la resa prestazionale dei contenuti multimediali.
- **Domain Layer:** definisce le entità di business pure del sistema (le classi modello come **User**, **PuppyPost** e **Notification**) e racchiude la logica applicativa indipendente dalle librerie esterne. Regola le operazioni principali del dominio, quali il filtraggio degli annunci e la gestione dei preferiti.
- **Presentation Layer:** contiene l'interfaccia utente visiva organizzata in *Activities* (**HomeActivity**, **SignInActivity**, **SignUpActivity**) e *Fragments* (**HomeFragment**, **PuppyDetailsFragment**, **ProfileFragment**, **FavoritesFragment**, **AddPuppyFragment**). L'interfaccia si avvale di componenti grafici dichiarati in layout XML e di *RecyclerView* affiancate da *Custom Adapters* (**FeedAdapter**, **SearchAdapter**, **FavoritesAdapter**). Gestisce l'interattività dell'utente e la risposta reattiva alle gesture native, come lo *Swipe-to-Favorite* implementato tramite **ItemTouchHelper**.

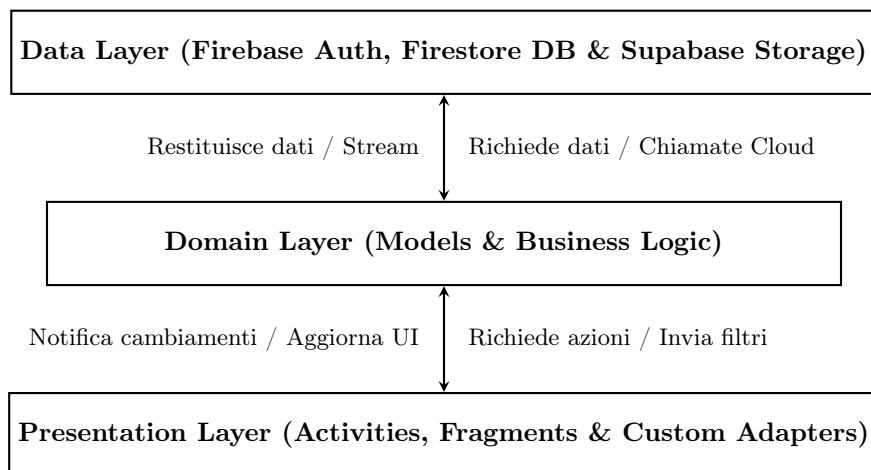


Figura 6: Architettura a tre livelli dell'applicazione Kotlin Paws

Mapa globale dell'architettura

Il diagramma riportato in Figura 7 illustra la mappa globale dell'architettura dell'applicazione Android **Paws**, evidenziando l'organizzazione strutturale dei moduli e le relazioni di dipendenza tra le diverse componenti del sistema:

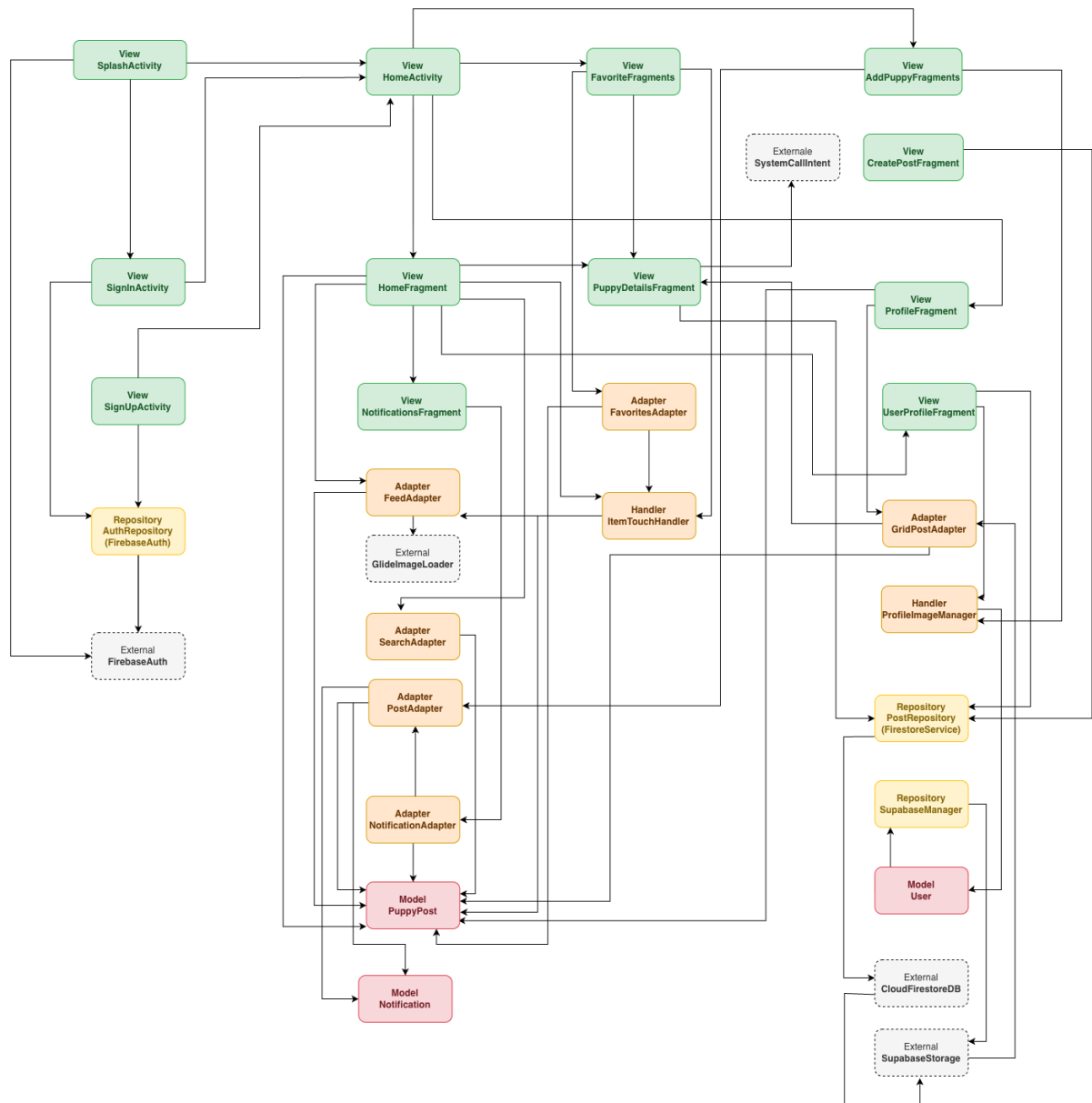


Figura 7: Diagramma delle componenti e mappa delle dipendenze dell'architettura Android Paws

Architettura d'avvio e navigazione

L'avvio dell'applicazione è guidato dalla **SplashActivity**, la quale verifica lo stato dell'autenticazione dell'utente tramite il client di **Firebase Authentication**:

- Se non viene rilevata alcuna sessione utente attiva (`currentUser == null`), il flusso viene reindirizzato alla schermata **SignInActivity**, dalla quale l'utente può eventualmente accedere alla schermata di registrazione **SignUpActivity**.
- Se la sessione è attiva (oppure a seguito del login/registrazione completati con successo), l'utente accede alla schermata principale **HomeActivity**.

- La **HomeActivity** ospita il contenitore principale dei contenuti ed abilita la navigazione tra le 4 schede della barra inferiore (**BottomNavigationView**):
 1. **Home** (**HomeFragment**): mostra il feed globale dei cuccioli, la barra di ricerca utenti.
 2. **Aggiungi Cucciolo** (**AddPuppyFragment**): permette la selezione dell'immagine dalla galleria nativa e l'apertura della sezione di creazione post (**CreatePostFragment**).
 3. **Preferiti** (**FavoritesFragment**): raccoglie l'elenco dei cuccioli salvati dall'utente per la consultazione rapida.
 4. **Profilo** (**ProfileFragment**): mostra i dettagli del profilo personale e la griglia dei post pubblicati dall'utente corrente.

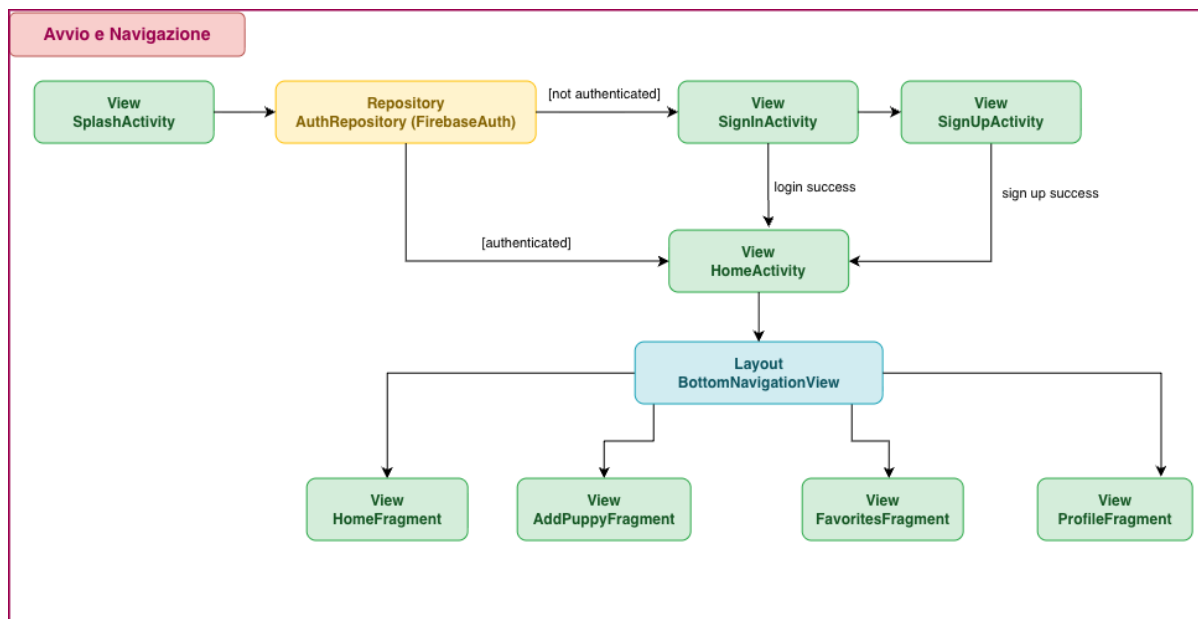


Figura 8: Mappa dell'architettura d'avvio e del flusso di navigazione dell'applicazione Paws

Architettura gestione utente e autenticazione

Le funzionalità relative all'identità utente, alla registrazione e alla personalizzazione del profilo sono strutturate secondo il seguente schema architetturale:

- **View:** le schermate `SignInActivity`, `SignUpActivity`, `ProfileFragment` e `UserProfileFragment` gestiscono l'input utente e la resa dell'interfaccia visiva.
- **Handler:** `ProfileImageManager` gestisce la selezione dell'immagine dell'utente prima dell'upload.
- **Repository:** `AuthRepository` gestisce la creazione dell'account e l'autenticazione remota su Firebase Auth, mentre `SupabaseManager` si occupa del caricamento sicuro dell'immagine profilo nel bucket di storage remoto.
- **Model & Database:** l'entità `User` modella i dati dell'utente, sincronizzati in tempo reale su Cloud Firestore.

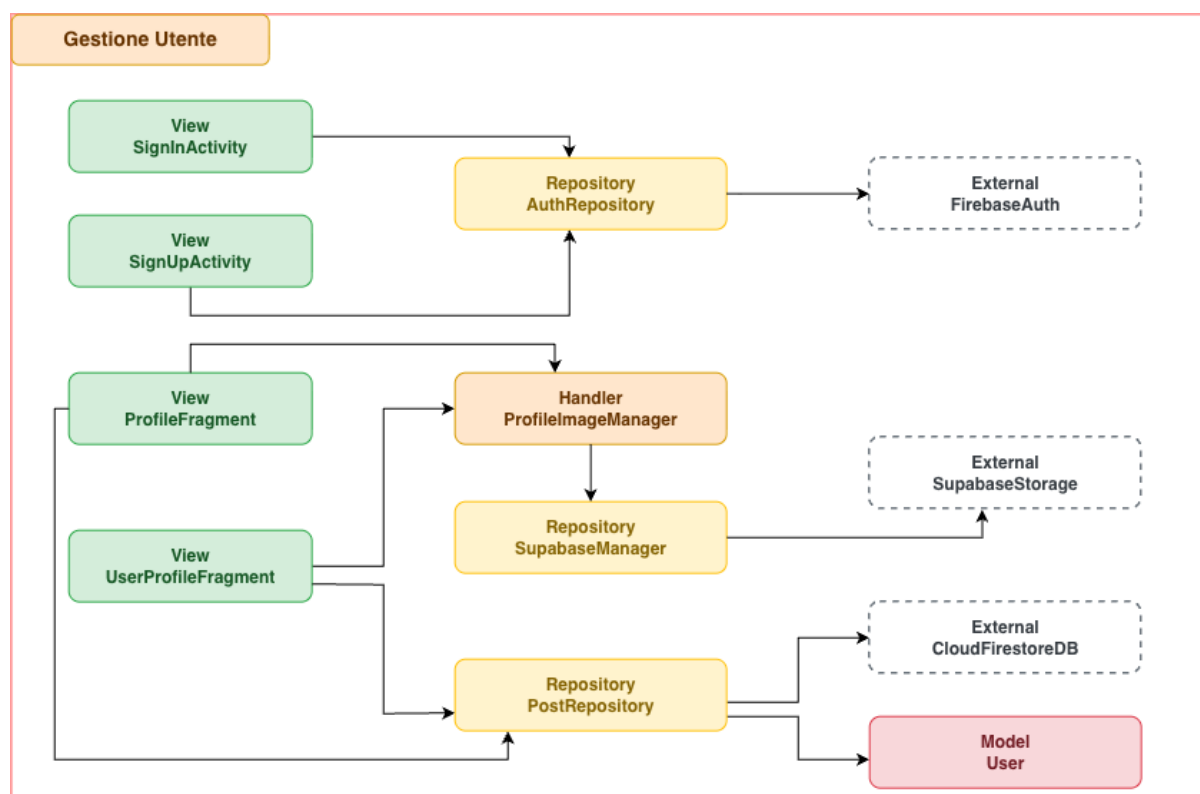


Figura 9: Architettura e flusso dati per la Gestione Utente e Autenticazione dell'applicazione Paws

Architettura gestione cuccioli e annunci

La gestione degli annunci dei cuccioli rappresenta il cuore dell'applicazione e sfrutta i seguenti moduli dedicati:

- **View:** HomeFragment, AddPuppyFragment, CreatePostFragment e PuppyDetailsFragment osservano i dati e reagiscono agli eventi dell'utente.
- **Adapters & Handlers:** FeedAdapter, SearchAdapter, PostAdapter e GridPostAdapter popolano le *RecyclerView*. L'*ItemTouchListener* intercetta le gesture di *swipe-to-favorite* per aggiungere istantaneamente un cucciolo ai preferiti dal feed.
- **Repository & Services:** PostRepository esegue le query NoSQL su **Cloud Firestore** per il recupero e filtraggio degli annunci, mentre **SupabaseManager** memorizza le foto dei cuccioli.
- **Model:** PuppyPost incapsula le informazioni del cucciolo (nome, razza, età, sesso, posizione e riferimento al proprietario).

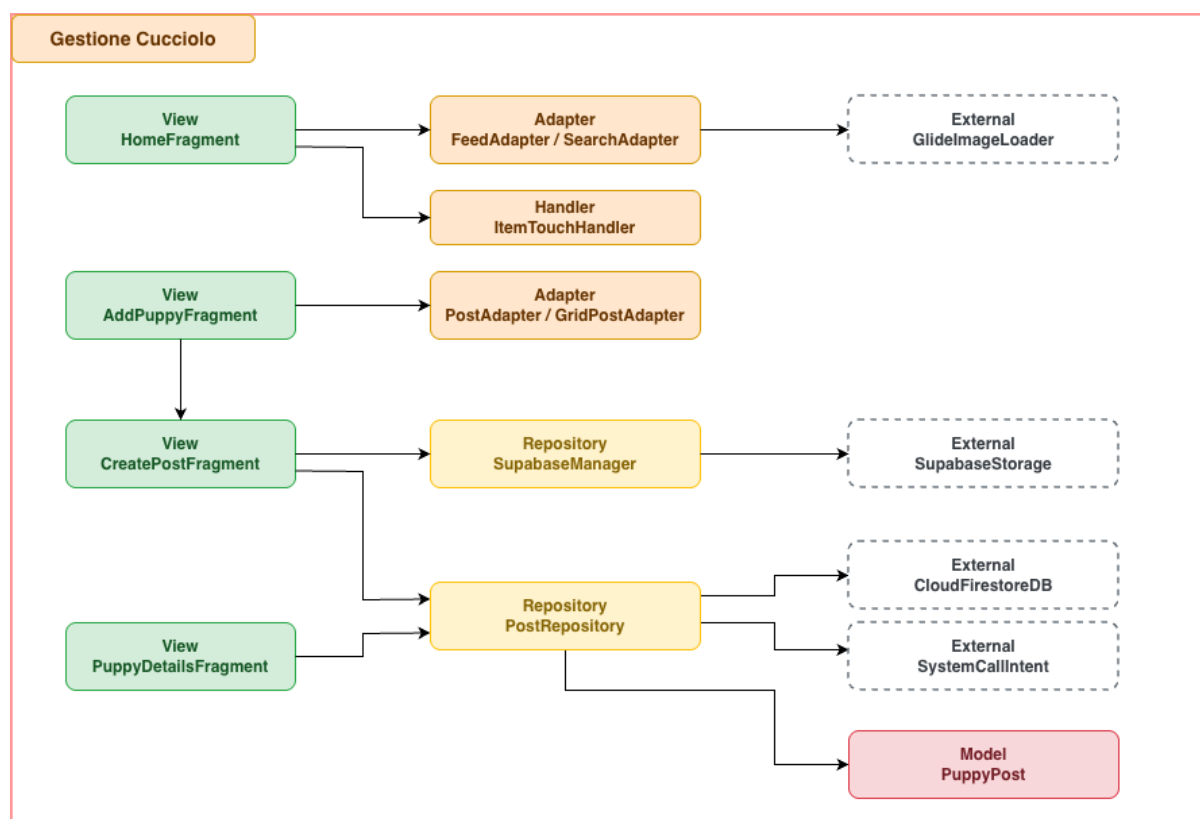


Figura 10: Architettura e flusso dati per la Gestione Cuccioli e Annunci dell'applicazione Paws

Architettura gestione notifiche e preferiti

La gestione dei preferiti e delle notifiche garantisce un'esperienza d'uso reattiva e sincronizzata:

- **View:** `FavoritesFragment` e `NotificationsFragment` mostrano rispettivamente la lista dei cuccioli salvati e le interazioni/notifiche ricevute.
- **Adapter:** `FavoritesAdapter` e `NotificationAdapter` formattano la resa grafica degli elementi. La libreria esterna **Glide** gestisce il caricamento fluido e asincrono delle immagini con caching in memoria.
- **Model:** `Notification` modella le notifiche di sistema salvate su Firestore.

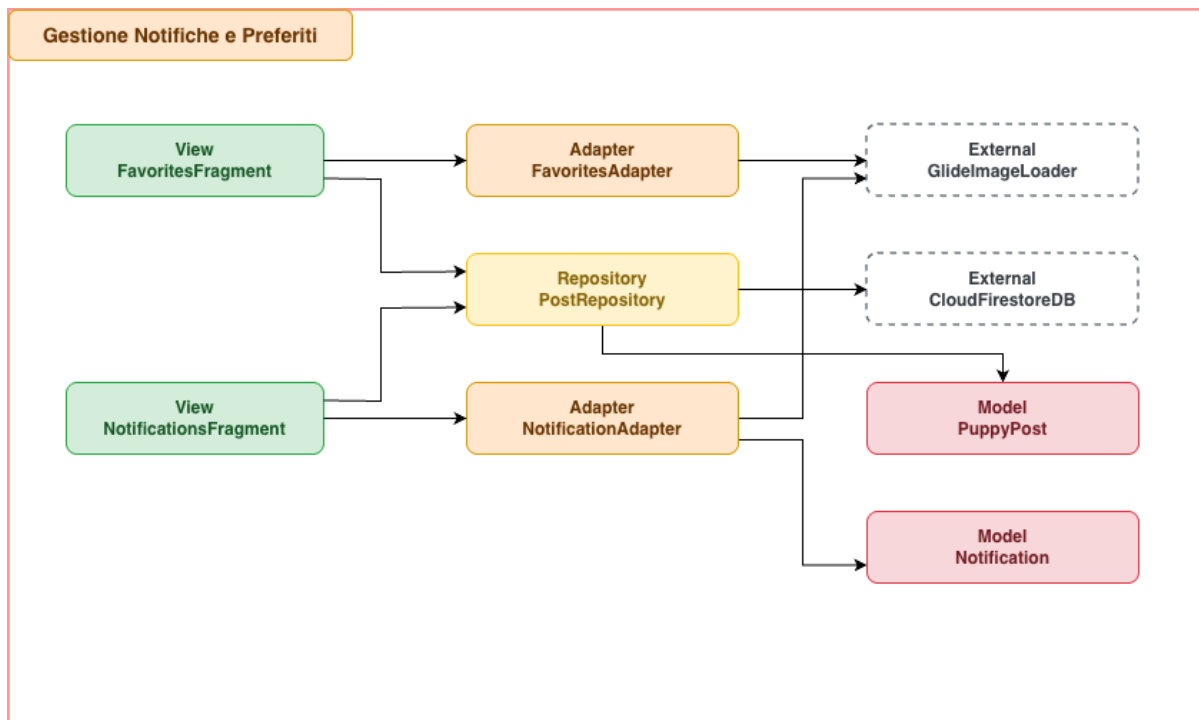


Figura 11: Architettura e flusso dati per la Gestione Cuccioli e Annunci dell'applicazione Paws

Mockup

Il mockup dell'applicativo GESTRi è stato realizzato con Figma, uno strumento per la progettazione e la prototipazione di interfacce utente. Ogni mockup rappresenta la versione progettuale dell'interfaccia, definendo struttura, elementi grafici e funzionalità prima della fase di sviluppo.

Verifica e Validazione del Software: Unit Tests

I seguenti test rappresentano l'insieme delle attività di verifica implementate per garantire il corretto funzionamento del progetto. Sono stati adottati due approcci complementari: test automatici lato codice e test manuali delle API.

Test Manuali

Per verificare le API è stata utilizzata l'applicazione Postman, uno strumento che consente di inviare richieste HTTP e analizzare le risposte. Questi test non rientrano nel diagramma degli Unit Test, ma rappresentano un'importante componente di validazione dell'integrazione e del comportamento delle API lato client-server.

Unit Test

La sezione presenta i test unitari eseguiti sul modulo Attività di GESTRi, con l'obiettivo di verificarne la correttezza funzionale e assicurare l'affidabilità del relativo codice.

```
1 class AttivitaIntegrationTests(TestCase):
2     def setUp(self):
3         self.client = Client()
4         # create users
5         self.staff = Utente.objects.create(email='staff@example.com', ruolo=
            Ruolo.STAFF)
6         self.staff.set_password('pass')
7         self.staff.save()
```

Unit Test 2: Attivita